
Ministerul Educației și cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 4

Dynamic programming

Elaborated:

std. gr. FAF-231

Nicolae MARGA

Verified:

asist. univ.

Cristofor FIȘTIC

Chișinău – 2025

Contents

1	Algorithm Analysis	3
1.1	Objective	3
1.2	Tasks	3
1.3	Theoretical Notes	3
1.3.1	Shortest Path Problem	3
1.3.2	Graph Classification by Connectivity and Density	3
1.4	Introduction to Shortest Path Algorithms	4
1.4.1	Floyd-Warshall Algorithm	4
1.4.2	Dijkstra's Algorithm	4
2	Implementation	4
2.1	Environment	4
2.2	Algorithm Implementations	5
2.2.1	Floyd-Warshall Implementation	5
2.2.2	Dijkstra's Implementation	5
2.3	Graph Generation	6
2.4	Measurement Approach	6
3	Results	8
3.1	Execution Time Analysis	8
3.2	Memory Usage Analysis	9
4	Observations	11
5	Conclusion	11
6	Repository	12

1 Algorithm Analysis

1.1 Objective

Study and analyze Floyd-Warshall and Dijkstra's shortest path algorithms. Perform empirical analysis comparing their performance across different graph types and sizes.

1.2 Tasks

- Implement Floyd-Warshall (all-pairs shortest path) and Dijkstra's (single-source shortest path) algorithms
- Establish the properties of the input data against which the analysis is performed
- Choose metrics for comparing algorithms
- Perform empirical analysis of the proposed algorithms
- Make a graphical presentation of the data obtained
- Make a conclusion on the work done

1.3 Theoretical Notes

Graphs are mathematical structures used to model relationships between objects, consisting of nodes (vertices) connected by edges. In weighted graphs, each edge has an associated cost or weight, which represents the distance, time, or some other metric between two connected vertices.

1.3.1 Shortest Path Problem

The shortest path problem involves finding a path between two vertices in a graph such that the sum of the weights of its constituent edges is minimized. This problem has numerous practical applications including route planning and navigation systems, network routing protocols, resource allocation optimization, social network analysis, and circuit design and optimization

1.3.2 Graph Classification by Connectivity and Density

Graphs can be classified in several ways. Based on edge density, we distinguish:

- Sparse graphs, which have relatively few edges compared to the number of vertices
- Dense graphs, which have a high number of edges approaching the maximum possible

The density of a graph significantly impacts the performance characteristics of shortest path algorithms. In this report, we focus on two primary graph types:

- Sparse graphs: With approximately $n - 1$ edges, where n is the number of vertices
- Dense graphs: Fully connected graphs with $\frac{n(n-1)}{2}$ edges for undirected graphs

1.4 Introduction to Shortest Path Algorithms

This report focuses on two fundamental shortest path algorithms:

1.4.1 Floyd-Warshall Algorithm

The Floyd-Warshall algorithm finds the shortest paths between all pairs of vertices in a graph. It uses dynamic programming to iteratively improve path estimates between vertices.

The algorithm computes shortest paths between all pairs of vertices in a single execution. Works with positive or negative edge weights (but no negative cycles). Has a time complexity of $\mathcal{O}(V^3)$ where V is the number of vertices. Has a space complexity of $\mathcal{O}(V^2)$

1.4.2 Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest paths from a single source vertex to all other vertices in a graph with non-negative edge weights. Key characteristics include: Computes shortest paths from a single source to all other vertices. Only works with non-negative edge weights. Has a time complexity of $\mathcal{O}((V + E) \log V)$ when implemented with a binary heap, where V is the number of vertices and E is the number of edges. Has a space complexity of $\mathcal{O}(V)$

2 Implementation

2.1 Environment

The project was implemented in Python using the following libraries:

- `networkx` for graph generation and manipulation
- `matplotlib` for visualization
- `pandas` for data organization and analysis
- `heapq` for priority queue implementation in Dijkstra's algorithm
- `tracemalloc` for memory usage measurement

-
- time for execution time measurement
 - random for generating random edge weights and graph structures

2.2 Algorithm Implementations

2.2.1 Floyd-Warshall Implementation

```
def floyd_warshall(graph):
    nodes = list(graph.keys())
    INF = float('inf')
    # Initialize distance matrix with infinity
    dist = {node: {other: INF for other in nodes} for node in nodes}
    # Set initial distances based on graph edges
    for u in graph:
        for v in graph[u]:
            dist[u][v] = graph[u][v]
        dist[u][u] = 0 # Distance to self is 0

    # Core Floyd-Warshall algorithm with triple nested loop
    for k in nodes:
        for i in nodes:
            for j in nodes:
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    return dist
```

2.2.2 Dijkstra's Implementation

```
def dijkstra(graph, start):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    visited = set()
    priority_queue = [(0, start)]
    while priority_queue:
        current_distance, current_node = heapq.heappop(priority_queue)

        # Skip if node already processed
        if current_node in visited:
            continue
```

```

visited.add(current_node)

# Process all neighbors of current node
for neighbor, weight in graph[current_node].items():
    distance = current_distance + weight
    if distance < distances[neighbor]:
        distances[neighbor] = distance
        heapq.heappush(priority_queue, (distance, neighbor))

return distances

```

2.3 Graph Generation

Two types of graphs were generated for this analysis:

```

def generate_sparse_graph(n):
    return nx.gnm_random_graph(n, n-1, seed=random.randint(1, 1000))
def generate_dense_graph(n):
    return nx.gnm_random_graph(n, n*(n-1)//2, seed=random.randint(1, 1000))

```

2.4 Measurement Approach

The following metrics were measured for each algorithm:

- Execution time (in seconds)
- Memory usage (in kilobytes)

For fair comparison between Floyd-Warshall (all-pairs) and Dijkstra's (single-source), Dijkstra's algorithm was executed from each possible source vertex and the total time and memory usage were recorded.

```

def measure_algorithm_performance(graph, algorithm, start_node=None):
    tracemalloc.start()
    start_time = time.perf_counter()
    if algorithm == 'floyd_warshall':
        dist = floyd_warshall(graph)
    elif algorithm == 'dijkstra':
        dist = dijkstra(graph, start_node)

    end_time = time.perf_counter()

```

```
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
```

```
return (end_time - start_time), peak / 1024 # Time in seconds, memory in KB
```

Graphs of varying sizes (10, 25, 50, 100, 250 vertices) were tested to observe how algorithm performance scales with graph size.

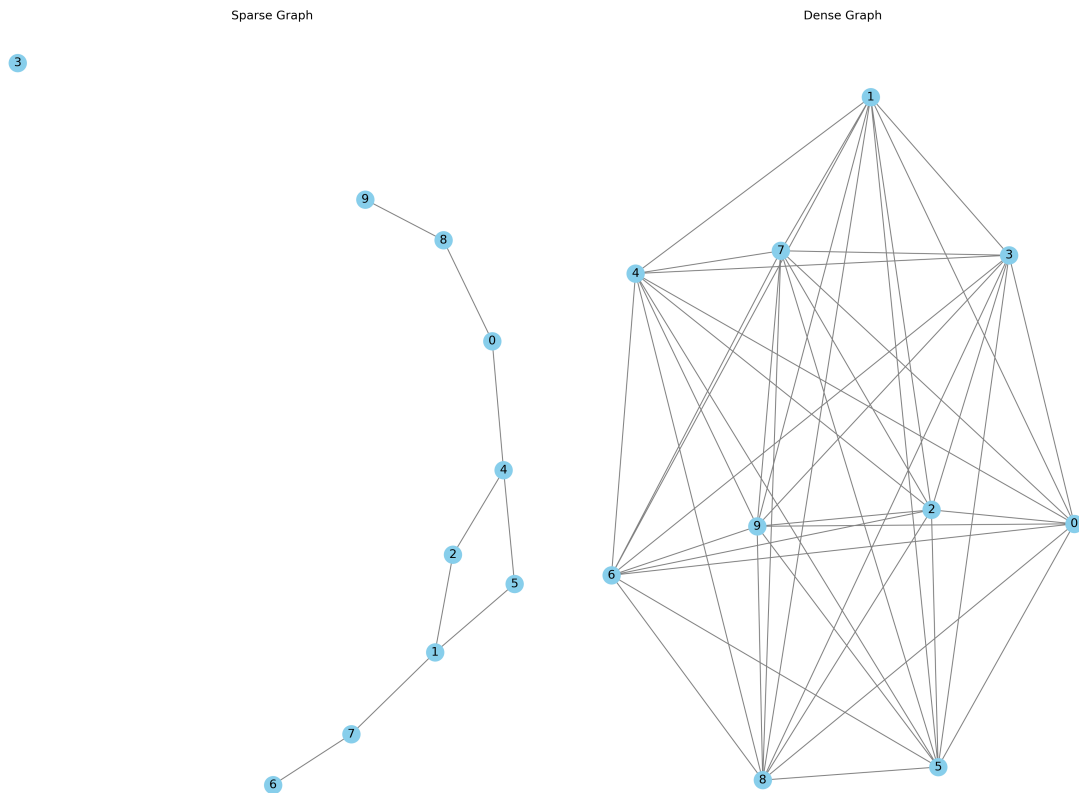


Figure 1: Sample visualizations of sparse (left) and dense (right) graph structures

3 Results

3.1 Execution Time Analysis

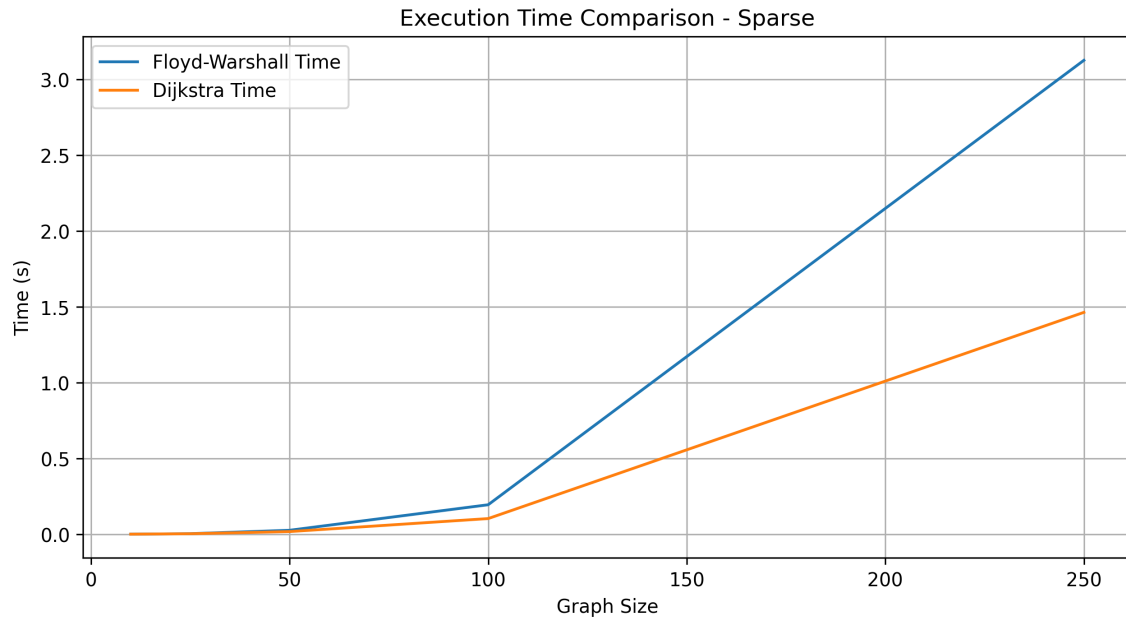


Figure 2: Execution time comparison between Floyd-Warshall and Dijkstra's algorithms on sparse graphs

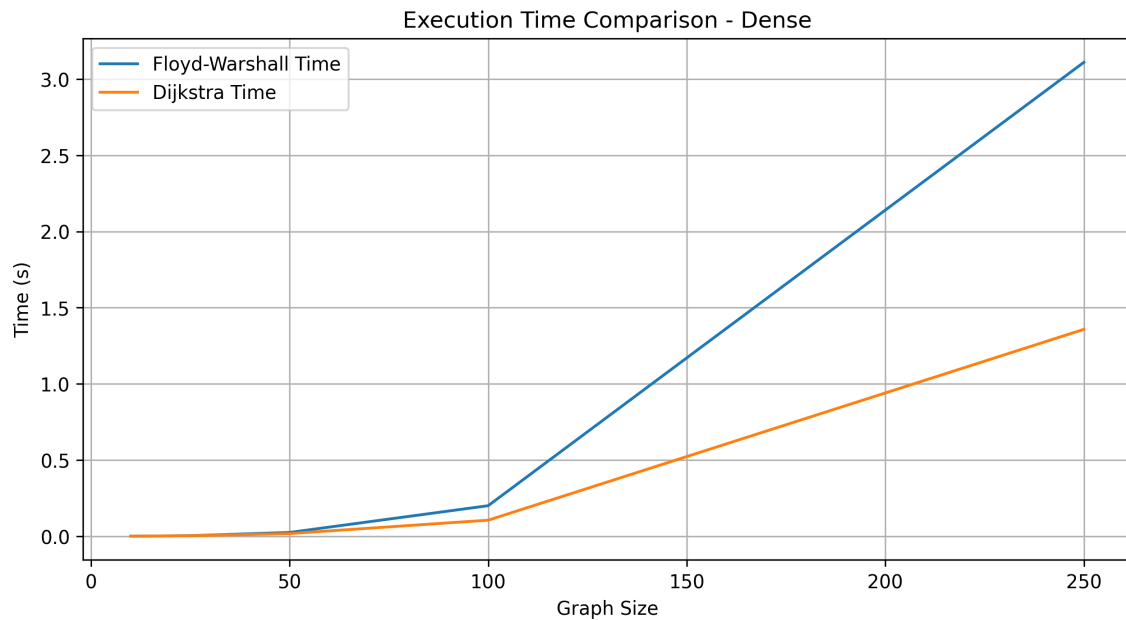


Figure 3: Execution time comparison between Floyd-Warshall and Dijkstra's algorithms on dense

3.2 Memory Usage Analysis

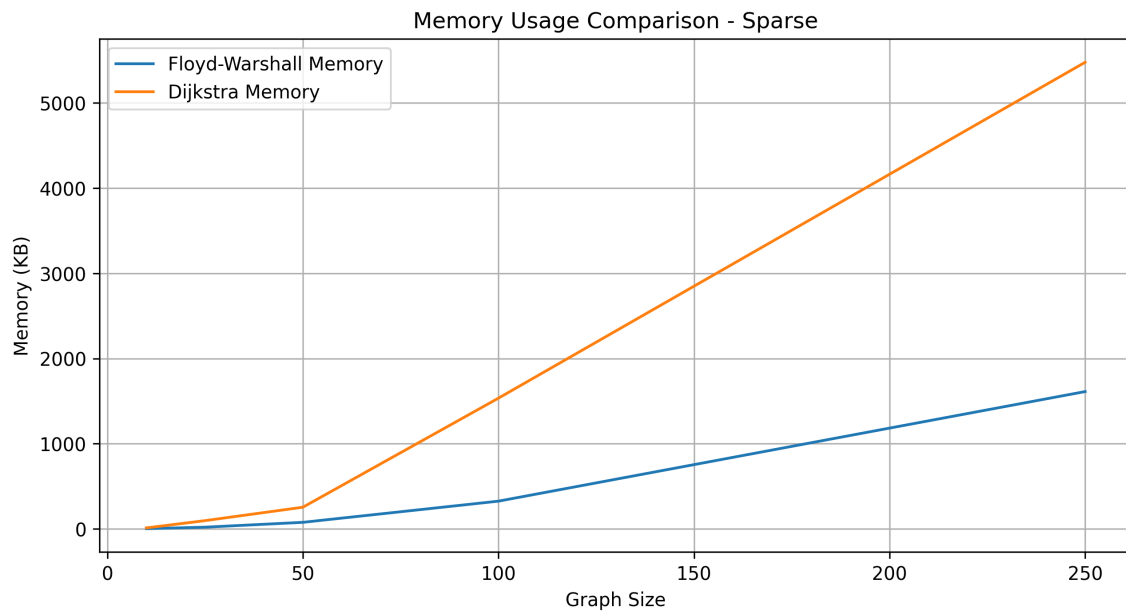


Figure 4: Memory usage comparison between Floyd-Warshall and Dijkstra's algorithms on sparse graphs

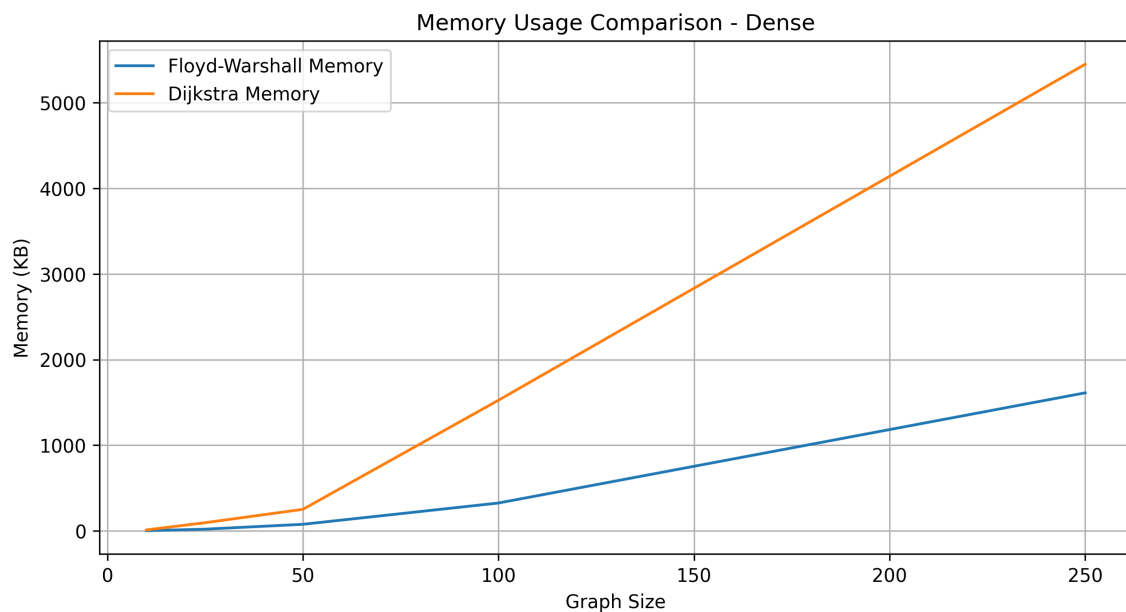


Figure 5: Memory usage comparison between Floyd-Warshall and Dijkstra's algorithms on dense graphs

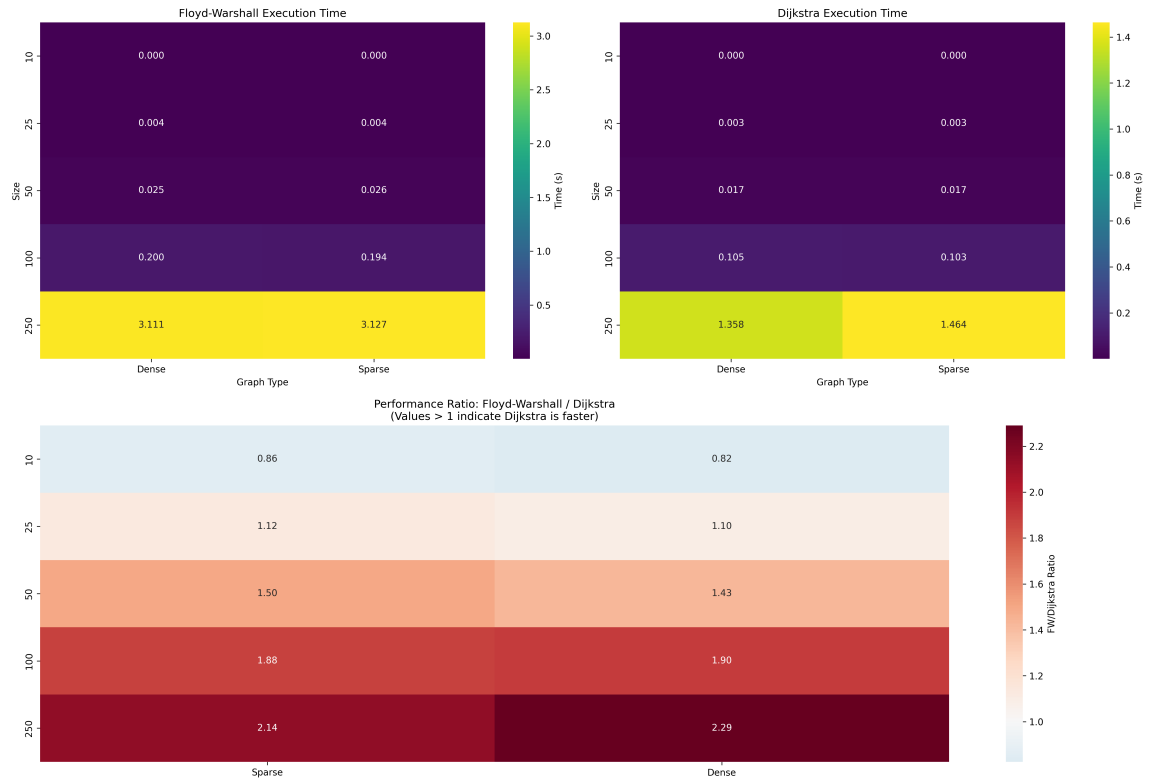


Figure 6: Time Comparison Heatmap

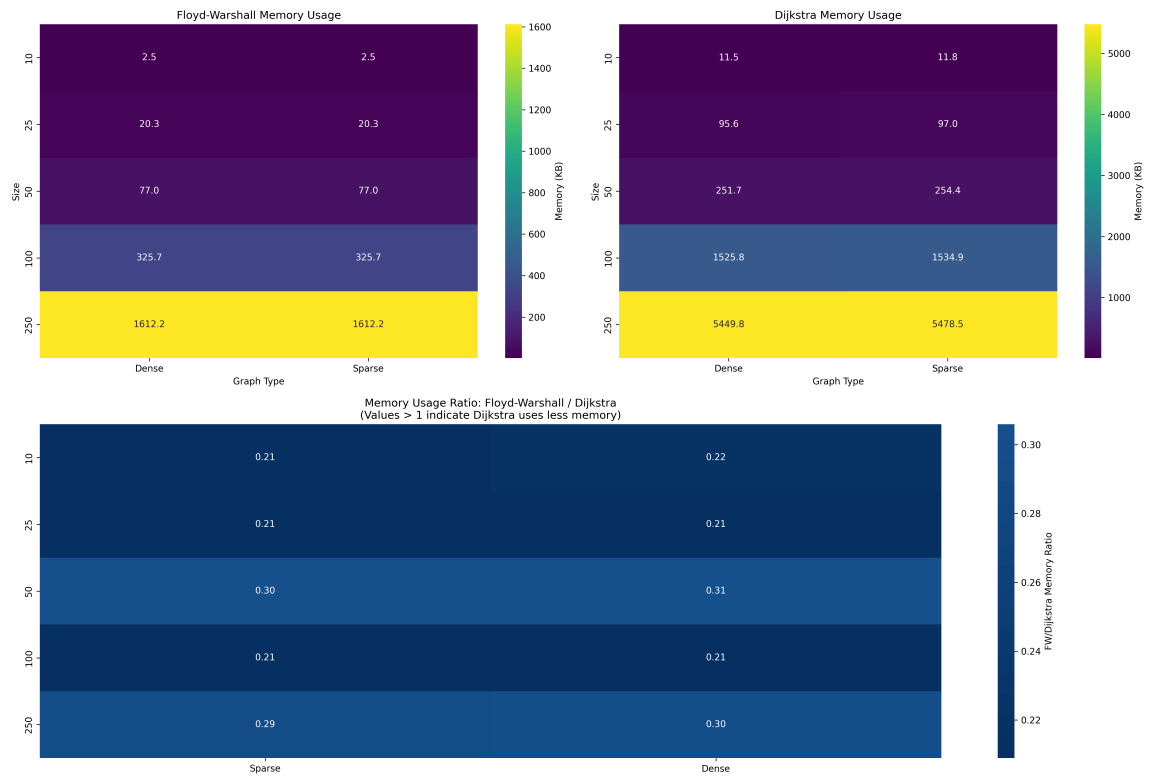


Figure 7: Memory Usage Comparison Heatmap

4 Observations

- **Time Complexity Validation:** The results confirm the expected time complexity behaviors:
 - Floyd-Warshall shows cubic growth $\mathcal{O}(V^3)$ with respect to graph size, making it less efficient for larger graphs.
 - Dijkstra’s algorithm (run from all possible sources) shows better scaling behavior, especially on sparse graphs.
- **Graph Density Impact:**
 - On **sparse graphs**, Dijkstra’s algorithm significantly outperforms Floyd-Warshall for larger graph sizes due to its ability to exploit the sparsity of edges.
 - On **dense graphs**, the performance gap between the algorithms narrows, but Dijkstra’s still maintains an advantage for the tested graph sizes.
- **Memory Usage:**
 - Floyd-Warshall consistently requires more memory due to its $\mathcal{O}(V^2)$ space complexity for storing the distance matrix.
 - Dijkstra’s algorithm shows more efficient memory usage, particularly on sparse graphs.
- **Crossover Points:**
 - For very small graphs ($n < 25$), the performance difference between the algorithms is minimal.
 - As graph size increases, the performance gap widens substantially, making algorithm choice more critical.

5 Conclusion

The empirical analysis of Floyd-Warshall and Dijkstra’s shortest path algorithms shows performance characteristics that are similar to their theoretical complexity analyses. Floyd-Warshall provides a simple solution for all-pairs shortest paths in a single execution but scales poorly with graph size due to its cubic time complexity. Dijkstra’s algorithm, while requiring multiple executions for all-pairs computation, demonstrates superior performance for larger graphs, especially sparse ones.

The choice between these algorithms should be guided by problem requirements, graph characteristics, and resource constraints. These results can help in algorithm selection for real-world applications such as network routing, transportation planning, and resource allocation optimization.

6 Repository

The source code for this laboratory work can be found here:

https://github.com/xnikug/AA_Labs/tree/lab4